UNIT IV – MULTIMODAL GENERATIVE AI APPLICATIONS

QUESTION 4 – MULTIPLE IMAGES FROM DIFFERENT TEXT PROMPTS

Question	Create multiple images from different text prompts and compare how changes in the prompts affect the generated images.
Folder Name	Q4_Multiple_Image_Comparison
Python File	text_to_image_compare.py
Application	Streamlit + Hugging Face Inference API

1. QUESTION

Create multiple images from different text prompts and compare how changes in the prompts affect the generated images.

2. AIM

To generate multiple engineering-related images using different text prompts and analyze how changes in the engineering subject, environment, level of detail and descriptive wording influence the generated results.

3. OBJECTIVES

- Create multiple images from different engineering-related prompts.
- Use a pre-trained text-to-image generative AI model.
- Compare the visual outputs produced by different prompts.
- Identify how changes in prompt content affect the generated image.
- Demonstrate prompt engineering through an interactive Streamlit application.

4. TOOLS AND TECHNOLOGIES

Technology	Purpose	Use in Q4
Python	Application development	Implements the image-generation application
Streamlit	Web interface	Accepts prompts and displays generated images
Hugging Face Inference API	AI inference	Provides access to the pre-trained text-to-image model
InferenceClient	API communication	Sends prompts and receives generated images
Pillow	Image handling	Supports image display and processing

5. WORKING PRINCIPLE

Three different engineering prompts are entered into the application. Each prompt describes a different engineering domain. The prompts are sent independently to the pre-trained text-to-image model. The generated outputs are then displayed side-by-side so the effect of changing the prompt can be visually compared.

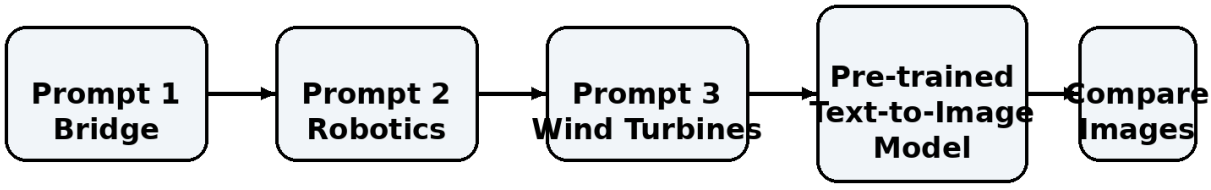

Figure 1. Corrected workflow for generating and comparing multiple prompt-based images.

6. PROMPTS USED IN THE ACTUAL DEMONSTRATION

Prompt	Engineering Area	Prompt Description
Prompt 1	Civil Engineering	A modern suspension bridge over a wide river, daylight, realistic civil engineering structure, steel cables, detailed architecture
Prompt 2	Robotics	An advanced industrial robotic arm working inside a modern engineering factory, metallic components, automation, realistic, high detail
Prompt 3	Renewable Energy	A large offshore wind turbine farm with modern engineering infrastructure, ocean environment, realistic engineering photography, daylight

7. STEP-BY-STEP IMPLEMENTATION

Step 1: Create the Q4_Multiple_Image_Comparison folder inside UNIT_IV_Multimodal_Generative_AI.

Step 2: Create the text_to_image_compare.py Python file.

Step 3: Install the required Streamlit and Hugging Face client libraries.

Step 4: Create and configure the Hugging Face access token as HF_TOKEN.

Step 5: Initialize the Hugging Face InferenceClient.

Step 6: Create three text input areas for the three engineering prompts.

Step 7: Send each prompt separately to the pre-trained text-to-image model.

Step 8: Receive the generated image for each prompt.

Step 9: Display the three outputs side-by-side.

Step 10: Compare the images based on subject, composition, environment, engineering details and visual style.

Step 11: Record the observations and conclude how prompt changes affected the generated results.

8. FOLDER AND FILE STRUCTURE

```
UNIT_IV_Multimodal_Generative_AI
└── Q4_Multiple_Image_Comparison
    ├── text_to_image_compare.py
    └── generated_images
        ├── prompt1_bridge.png
        ├── prompt2_robotic_arm.png
        └── prompt3_wind_turbines.png
```

9. COMPLETE IMPLEMENTATION CODE

```
import streamlit as st
from huggingface_hub import InferenceClient
import os

st.set_page_config(
    page_title="Engineering Prompt Comparison",
    page_icon="🎨",
    layout="wide"
)

st.title("🎨 Engineering Image Prompt Comparison")

st.write(
    "Generate multiple engineering images from different prompts "
    "and compare how prompt changes affect the generated results."
)

HF_TOKEN = os.getenv("HF_TOKEN")

if not HF_TOKEN:
    st.error("HF_TOKEN is not configured.")
    st.stop()

client = InferenceClient(api_key=HF_TOKEN)

prompt1 = st.text_area(
    "Prompt 1 – Civil Engineering",
    "A modern suspension bridge over a wide river, daylight, "
    "realistic civil engineering structure, steel cables, detailed architecture"
)

prompt2 = st.text_area(
    "Prompt 2 – Robotics",
    "An advanced industrial robotic arm working inside a modern "
    "engineering factory, metallic components, automation, realistic, high detail"
)

prompt3 = st.text_area(
    "Prompt 3 – Renewable Energy",
    "A large offshore wind turbine farm with modern engineering "
    "infrastructure, ocean environment, realistic engineering photography, daylight"
)

if st.button("🎨 Generate All Three Images"):
    prompts = [
        ("Civil Engineering", prompt1),
        ("Robotics", prompt2),
        ("Renewable Energy", prompt3)
    ]

    images = []
```

```
    try:
        with st.spinner("Generating three engineering images..."):
            for title, prompt in prompts:
                image = client.text_to_image(
                    prompt=prompt,
                    model="black-forest-labs/FLUX.1-schnell"
                )
                images.append((title, image))

        st.success("All three images generated successfully!")

        col1, col2, col3 = st.columns(3)

        with col1:
            st.subheader("Image 1")
            st.write("Civil Engineering")
            st.image(images[0][1], caption="Suspension Bridge", width="stretch")

        with col2:
            st.subheader("Image 2")
            st.write("Robotics")
            st.image(images[1][1], caption="Industrial Robotic Arm", width="stretch")

        with col3:
            st.subheader("Image 3")
            st.write("Renewable Energy")
            st.image(images[2][1], caption="Offshore Wind Turbines", width="stretch")

        st.divider()
        st.subheader("Prompt Comparison")
        st.write("Prompt 1 focuses on civil engineering and structural bridge features.")
        st.write("Prompt 2 changes the engineering domain to industrial robotics.")
        st.write("Prompt 3 changes the domain to renewable energy and an offshore environment.")

    except Exception as e:
        st.error("Image generation failed.")
        st.write(str(e))
```

10. EXECUTION COMMAND

```
streamlit run ".\Q4_Multiple_Image_Comparison\text_to_image_compare.py"
```

11. IMPLEMENTATION SCREENSHOT

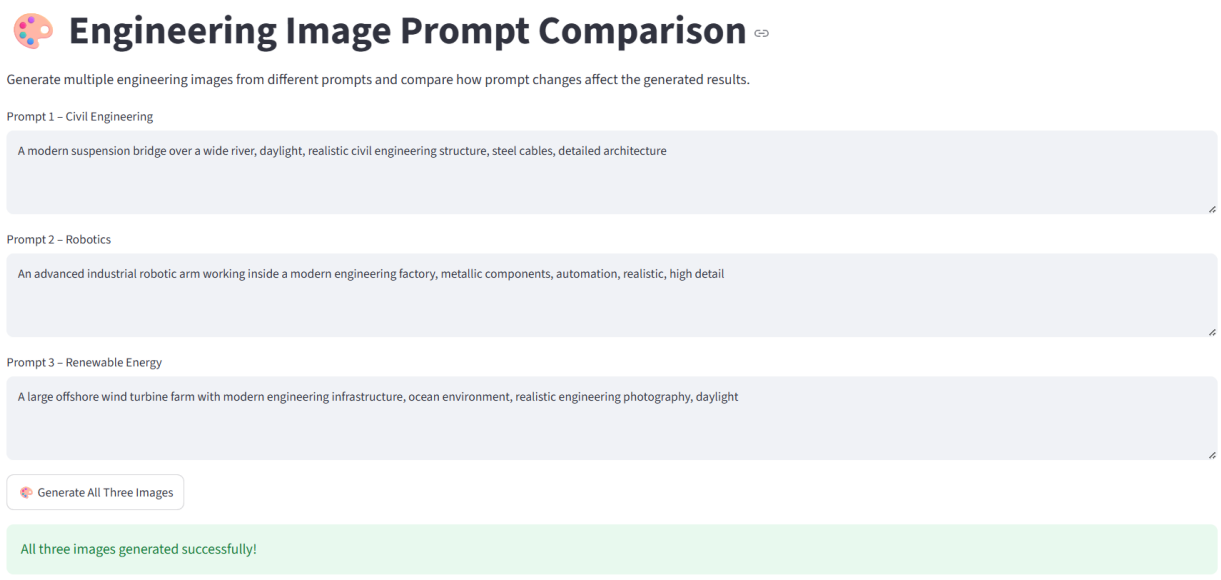

Figure 2. Q4 application showing three different engineering prompts and successful image generation.

12. GENERATED IMAGE OUTPUT

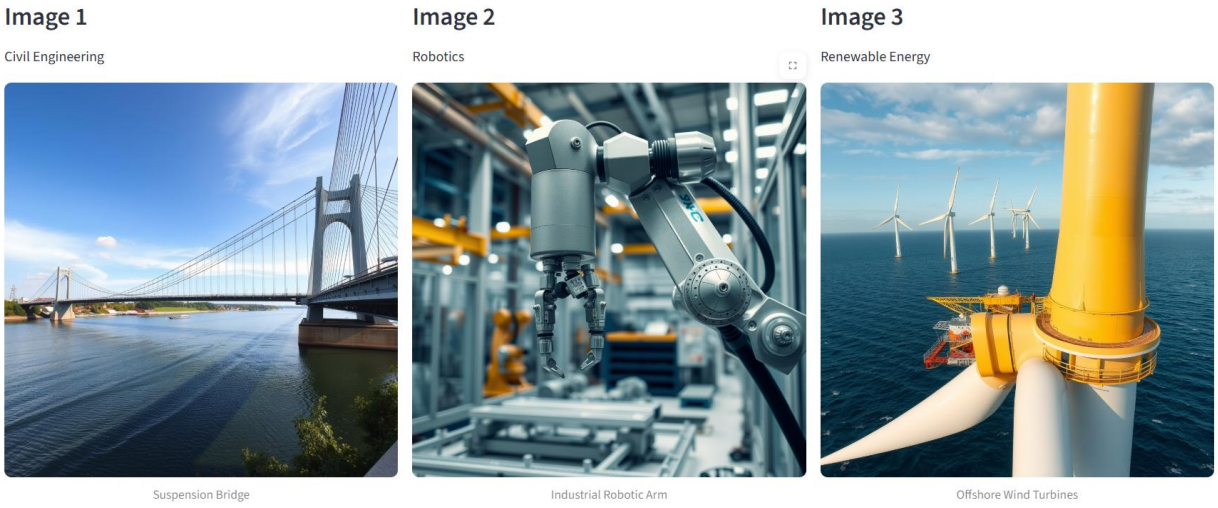

Figure 3. Q4 output showing three clearly different generated images: suspension bridge, industrial robotic arm and offshore wind turbines.

13. OBSERVATION AND COMPARISON

The screenshots provide a direct comparison of the three generated outputs. Each prompt contains a different engineering subject, so the model produces visibly different content. The first prompt produces a civil-engineering bridge scene, the second produces an industrial robotic system, and the third produces an offshore renewable-energy scene.

Aspect	Prompt 1 – Bridge	Prompt 2 – Robotics	Prompt 3 – Wind Turbines

Main subject	Suspension bridge	Industrial robotic arm	Offshore wind turbines
Environment	Wide river and open sky	Modern engineering factory	Ocean and offshore infrastructure
Engineering domain	Civil engineering	Robotics / automation	Renewable energy
Visual emphasis	Cables, towers, bridge structure	Mechanical joints, metallic components	Turbine tower, blades and marine structure
Prompt effect	Structural scene	Machine-focused scene	Energy infrastructure scene

14. HOW PROMPT CHANGES AFFECT THE IMAGES

- Changing the main noun changes the generated subject. 'Suspension bridge', 'robotic arm' and 'wind turbine farm' produce different engineering scenes.
- Adding environment terms changes the background and context. River, factory and ocean descriptions create different settings.
- Adding descriptive terms such as 'realistic', 'high detail' and 'detailed architecture' guides the visual appearance and detail level.
- Adding engineering-specific components helps the model emphasize relevant structures such as cables, mechanical joints, turbine blades and support systems.
- Therefore, prompt wording acts as a control mechanism for the image-generation process.

15. RESULT

The multiple-image experiment was successfully completed. Three different engineering prompts produced three visibly different outputs. The results clearly demonstrate that changing the engineering subject and adding contextual descriptions changes the content, environment and visual emphasis of the generated images.

16. ADVANTAGES

- Clearly demonstrates prompt engineering.
- Allows side-by-side comparison of multiple generated images.
- Can be used for civil, mechanical, robotics, electrical and renewable-energy concepts.
- Provides rapid visual concept generation.
- Useful for engineering education, presentations and early design visualization.

17. LIMITATIONS

- Generated images are visual concepts and may not represent engineering-accurate designs.
- The result depends on the selected model and its capabilities.
- Different generations may vary even with similar prompts.
- More complex prompts can require longer generation time or higher inference cost.

18. FUTURE ENHANCEMENTS

- Allow users to save and download all generated images.
- Add prompt history and automatic comparison reports.
- Add image similarity or CLIP-based evaluation.
- Support multiple engineering prompt templates.

- Add CAD-style or technical-drawing generation for engineering education.

19. CONCLUSION

This experiment successfully demonstrates how prompt changes influence text-to-image generation. The three actual outputs show clear differences between civil engineering, robotics and renewable-energy applications. The experiment confirms that selecting suitable nouns, engineering details, environments and descriptive terms can guide a pre-trained generative model toward different visual results.

20. VIVA / KEY POINTS

- A text-to-image model generates an image from a natural-language prompt.
- Prompt engineering means designing prompts to control the generated output.
- Changing the engineering subject produces a different image.
- Context words such as factory, river and ocean influence the generated environment.
- Detailed prompts generally provide more guidance to the model.